

BICYCLE RETAIL SALES

ANALYSIS REPORT

By,
S.Abinaya

CHAPTER 1

INTRODUCTION

Domain	Retail Sales & Customer Analytics
Institution	Vinsup Skill Academy
Project Guide	Mr.Athithya Jayakumar
Tools Used	Excel, MySQL, Python (Jupyter Notebook), Microsoft Power BI
Dataset	1,615 Orders / 4,722 Order Line Items / 1,445 Customers
Date	July 2026

1.1 Problem Statement

A bicycle retail business operating across multiple stores wants to evaluate its sales performance across stores, product categories, and brands to understand business growth and improve decision-making. The project focuses on converting raw, normalized database records into meaningful insights by cleaning, querying, and modeling the data. Sales and customer value are analyzed to identify top-performing stores, products, and staff, and to segment customers by engagement and value. An interactive Power BI dashboard is built to present these insights and support strategic business decisions.

1.2 Project Description

The business operates across 3 stores, selling products from 9 brands across 7 categories, served by 10 staff members, and purchased by a base of 1,445 registered customers. The management requires a complete analysis of sales performance to identify top-performing stores and products, understand customer value, and make data-driven decisions for marketing, staffing, and inventory planning.

This report documents the complete analysis performed on a relational sales dataset of 1,615 orders and 4,722 order line items, covering database querying, RFM-based customer segmentation, and interactive dashboard design — executed using MySQL, Python, and Microsoft Power BI.

1.4 Objectives of the Project

- Consolidate and query sales data stored across multiple related database tables.
- Analyse sales performance by store, product, category, brand, and staff.
- Segment customers using the RFM (Recency, Frequency, Monetary) model.
- Design an interactive Power BI dashboard to present business insights.

1.5 Outcomes of the Project

The outcome of this project is a validated, query-ready sales database and a segmented customer dataset. The project successfully analyzes sales performance across stores, product categories, and brands using SQL, and builds a customer segmentation model using Python. The Power BI dashboard provides clear visual insights into sales trends, top-performing stores and products, and customer value segments. This project demonstrates how a combined SQL + Python + Power BI workflow can support business decision-making, marketing strategy, and inventory planning.

1.6 Scope of the Project

This project focuses on analyzing retail sales data using SQL, Python, and Power BI to identify sales trends, store and product performance, and customer value segments. It helps the business make better decisions through direct querying, statistical segmentation, and interactive dashboard reporting.

1.7 Data Description

Table	Rows	Key Fields
customers	1,445	customer_id, name, city, state
orders	1,615	order_id, customer_id, order_date, store_id, staff_id
order_items	4,722	order_id, product_id, quantity, list_price, discount
products	321	product_id, product_name, brand_id, category_id
stores	3	store_id, store_name
staffs	10	staff_id, name, store_id
stocks	939	store_id, product_id, quantity

The dataset consists of 9 related tables — customers, orders, order_items, products, brands, categories, stores, staffs, and stocks — stored in a MySQL database named 'sales'. Order-level details include Order ID, Order Date, Store, Staff, and Status; line-item details include Product, Quantity, List Price, and Discount. This dataset is used for querying, RFM segmentation, and dashboard visualization.

CHAPTER 2

EXCEL DATA CLEANING & PREPARATION

2.1 Dataset Overview (Before Cleaning)

Table	Rows	Columns (Original)
brands	9	brand_id, brand_name
categories	7	category_id, category_name
customers	1,445	customer_id, first_name, last_name, phone, email, street, city, state, zip_code
orders	1,615	order_id, customer_id, order_status, order_date, required_date, shipped_date, store_id, staff_id
order_items	4,722	order_id, item_id, product_id, Category_id, category_name, quantity, list_price, discount, Total_price, Product_name
products	321	product_id, product_name, brand_id, category_id, model_year, list_price
staffs	10	staff_id, first_name, last_name, email, phone, active, store_id, manager_id
stocks	939	store_id, product_id, quantity
stores	3	store_id, store_name, phone, email, street, city, state, zip_code

Table1. counts and original column names for each table.

2.2 Standardize Column Headers

The column names across all worksheets such as customers, orders, products, order_items, stores, staffs, brands, categories, and stocks were standardized using a consistent naming convention. Column headers were converted into snake_case format to maintain uniformity and improve compatibility with SQL and Python.

2.3 Remove Duplicate Records

The datasets were checked for duplicate records using Excel's Remove Duplicates feature. Duplicate entries were identified and removed from relevant tables to ensure data accuracy and maintain data integrity before further analysis.

2.4 Handle Missing Values

The datasets were examined for missing values using Filter and Go To Special options. Blank values found in fields such as ZIP codes and phone numbers were replaced with 'Not Found' to maintain consistency and avoid issues during SQL import and analysis.

2.5 Data Type Conversion

Date columns such as order_date, required_date, and shipped_date were converted into Excel Date format. Numeric fields including quantity, list_price, and discount were formatted as numeric values to ensure accurate calculations and analysis.

2.6 Data Validation

The order_status column contained numeric codes (1–4). These codes were mapped to business-readable labels using a lookup list (Data Validation), improving data readability across all **1,615** orders.

Code	Label	Orders	Share
1	Pending	62	3.8%
2	Processing	63	3.9%
3	Rejected	45	2.8%
4	Completed	1,445	89.5%

All 1,615 orders mapped to a label with zero unmapped codes; the large majority (89.5%) are Completed.

2.7 Create New Derived Columns

Add a column in order_items to calculate $\text{total_price} = (\text{list_price} \times \text{quantity}) - \text{discount}$.

C	D	E	F	G	H	I	J
product_id	Category_id	category_name	quantity	list_price	discount	Total_price	Product_name
20	3	Cruisers Bicycles	1	599.99	0.2	599.79	Electra Townie Original 7D EQ - Women's - 2016
8	6	Mountain Bikes	2	1799.99	0.07	3599.91	Trek Remedy 29 Carbon Frameset - 2016
10	4	Cyclocross Bicycles	2	1549	0.05	3097.95	Surly Straggler - 2016
16	3	Cruisers Bicycles	2	599.99	0.05	1199.93	Electra Townie Original 7D EQ - 2016
4	6	Mountain Bikes	1	2899.99	0.2	2899.79	Trek Fuel EX 8 29 - 2016
20	3	Cruisers Bicycles	1	599.99	0.07	599.92	Electra Townie Original 7D EQ - Women's - 2016
16	3	Cruisers Bicycles	2	599.99	0.05	1199.93	Electra Townie Original 7D EQ - 2016
3	6	Mountain Bikes	1	999.99	0.05	999.94	Surly Wednesday Frameset - 2016
20	3	Cruisers Bicycles	1	599.99	0.05	599.94	Electra Townie Original 7D EQ - Women's - 2016
2	6	Mountain Bikes	2	749.99	0.1	1499.88	Ritchey Timberwolf Frameset - 2016
10	4	Cyclocross Bicycles	2	1549	0.05	3097.95	Surly Straggler - 2016
17	3	Cruisers Bicycles	1	429	0.07	428.93	Pure Cycles Vine 8-Speed - 2016
26	2	Comfort Bicycles	1	599.99	0.07	599.92	Electra Townie Original 7D EQ - 2016
18	3	Cruisers Bicycles	1	449	0.07	448.93	Pure Cycles Western 3-Speed - Women's - 2015/2016
12	3	Cruisers Bicycles	2	549.99	0.05	1099.93	Electra Townie Original 21D - 2016

total_price formula verified correct for all 4,722 order line items, with zero discrepancies.

2.8 Merge Lookup Data

Use XLOOKUP to merge product names into order_items using product_id.

J23 fx `=XLOOKUP(C23,Sheet1!A:A,Sheet1!B:B,"Not found",0)`

	C	D	E	F	G	H	I	J
	product_id	Category_id	category_name	quantity	list_price	discount	Total_price	Product_name
1	20	3	Cruisers Bicycles	1	599.99	0.2	599.79	Electra Townie Original 7D EQ - Women's - 2016
2	8	6	Mountain Bikes	2	1799.99	0.07	3599.91	Trek Remedy 29 Carbon Frameset - 2016
3	10	4	Cyclocross Bicycles	2	1549	0.05	3097.95	Surly Straggler - 2016
4	16	3	Cruisers Bicycles	2	599.99	0.05	1199.93	Electra Townie Original 7D EQ - 2016
5	4	6	Mountain Bikes	1	2899.99	0.2	2899.79	Trek Fuel EX 8 29 - 2016
6	20	3	Cruisers Bicycles	1	599.99	0.07	599.92	Electra Townie Original 7D EQ - Women's - 2016
7	16	3	Cruisers Bicycles	2	599.99	0.05	1199.93	Electra Townie Original 7D EQ - 2016
8	3	6	Mountain Bikes	1	999.99	0.05	999.94	Surly Wednesday Frameset - 2016
9	20	3	Cruisers Bicycles	1	599.99	0.05	599.94	Electra Townie Original 7D EQ - Women's - 2016
10	2	6	Mountain Bikes	2	749.99	0.1	1499.88	Ritchey Timberwolf Frameset - 2016
11	10	4	Cyclocross Bicycles	2	1549	0.05	3097.95	Surly Straggler - 2016
12	17	3	Cruisers Bicycles	1	429	0.07	428.93	Pure Cycles Vine 8-Speed - 2016
13	26	2	Comfort Bicycles	1	599.99	0.07	599.92	Electra Townie Original 7D EQ - 2016
14	18	3	Cruisers Bicycles	1	449	0.07	448.93	Pure Cycles Western 3-Speed - Women's - 2015/2016
15	12	3	Cruisers Bicycles	2	549.99	0.05	1099.93	Electra Townie Original 21D - 2016
16	20	3	Cruisers Bicycles	1	599.99	0.1	599.89	Electra Townie Original 7D EQ - Women's - 2016
17	3	6	Mountain Bikes	2	999.99	0.07	1999.91	Surly Wednesday Frameset - 2016
18	9	5	Electric Bikes	2	2999.99	0.07	5999.91	Trek Conduitt+ - 2016
19	15	3	Cruisers Bicycles	1	529.99	0.07	529.92	Electra Moto 1 - 2016

product_name, category_name and brand_name all merged with a 100% match rate — no unmatched product_id values.

2.9 Create Basic Pivot Table

A pivot table was built to summarize the 4,722 line items into category-level revenue:

Category	Total Sales (\$)
Mountain Bikes	327803.83
Road Bikes	438452.02
Cruisers Bicycles	1109007.23
Electric Bikes	799846.1
Cyclocross Bicycles	1020214.56
Comfort Bicycles	3030651.45
Children Bicycles	1852516.12

2.10 Sort and Filter for Outliers

Find outliers in pricing .e.g., products with very high list_price

Product	List Price (\$)
Trek Domane SLR 9 Disc - 2018	11,999.99
Trek Domane SLR 8 Disc - 2018	7,499.99
Trek Silque SLR 8 Women's - 2017	6,499.99
Trek Domane SL Frameset - 2018	6,499.99
Trek Domane SL Frameset Women's - 2018	6,499.99

2.11 Prepare Final CSVs

Save cleaned sheets as CSVs (customers.csv, orders.csv, etc.) ready for SQL import.

File	Rows
brands.csv	9
categories.csv	7
customers.csv	1,445
orders.csv	1,615
order_items.csv	4,722
products.csv	321
staffs.csv	10
stocks.csv	939

CHAPTER 3

DATABASE DESIGN & SQL ANALYSIS

3.1 Database Schema & Relationships

The database follows a normalized retail design. Core relationships:

- orders → linked to customers and stores
- order_items → linked to orders and products
- products → linked to brands and categories
- stocks → linked to products and stores

3.2 Data Cleaning & Validation

- Checked for duplicate store_id entries using GROUP BY / HAVING COUNT(*) > 1.
- Added an index on staffs.store_id to improve join performance.
- Reviewed table structures (SHOW CREATE TABLE) for stocks, stores, and staffs before analysis.

3.3 Business Queries — Store & Product Performance

Store-wise Revenue

```
SELECT s.store_name,  
       ROUND(SUM((oi.list_price*oi.quantity)*(1-oi.discount)),2) AS total_sales  
FROM stores s  
JOIN orders o ON s.store_id = o.store_id  
JOIN order_items oi ON o.order_id = oi.order_id  
GROUP BY s.store_name ORDER BY total_sales DESC;
```

Output: Calculates real revenue per store after discount, ranked highest to lowest. Identifies the best and worst performing store locations.

Top 5 Best-Selling Products

```
SELECT p.product_id, p.product_name, SUM(oi.quantity) AS total_quantity_sold  
FROM order_items oi JOIN products p ON oi.product_id = p.product_id  
GROUP BY p.product_id, p.product_name  
ORDER BY total_quantity_sold DESC LIMIT 5;
```

Output: Sums quantity sold per product and lists the top 5 by volume. Guides inventory and restocking priorities.

3.4 Business Queries — Customer & Staff Analysis

Customer Value Report

```
SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
       COUNT(DISTINCT o.order_id) AS total_orders,
       ROUND(SUM((oi.list_price*oi.quantity)*(1-oi.discount)),2) AS total_revenue
FROM customers c JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY total_revenue DESC;
```

Output: Ranks every customer by orders placed and total revenue generated. Identifies the most valuable customers for loyalty/VIP targeting.

Staff Performance Report

```
SELECT s.staff_id, CONCAT(s.first_name, ' ', s.last_name) AS staff_name,
       COUNT(DISTINCT o.order_id) AS total_orders,
       ROUND(SUM((oi.list_price*oi.quantity)*(1-oi.discount)),2) AS total_revenue
FROM staffs s JOIN orders o ON s.staff_id = o.staff_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY s.staff_id, s.first_name, s.last_name
ORDER BY total_revenue DESC;
```

Output: Ranks staff by number of orders handled and revenue generated. Supports staff recognition and coaching decisions.

3.5 Low-Stock Alert Query

Low-Stock Alert Report

```
SELECT s.store_id, st.store_name, p.product_id, p.product_name, s.quantity
FROM stocks s JOIN products p ON s.product_id = p.product_id
JOIN stores st ON s.store_id = st.store_id
WHERE s.quantity < 10 ORDER BY s.quantity ASC;
```

Output: Flags every product, per store, with stock below 10 units. An action-ready report to prevent stockouts of fast-moving products.

Customer Spend Segmentation (SQL)

```
SELECT c.customer_id, ROUND(SUM((oi.list_price*oi.quantity)*(1-oi.discount)),2) AS
total_spent,
CASE
  WHEN SUM(...) < 2000 THEN 'Low'
  WHEN SUM(...) BETWEEN 2000 AND 5000 THEN 'Medium'
  ELSE 'High'
END AS spending_segment
FROM customers c JOIN orders o ... JOIN order_items oi ...
GROUP BY c.customer_id ...;
```

Output: Classifies each customer as Low, Medium, or High spender. A quick, direct-SQL alternative to statistical segmentation.

CHAPTER 3

PYTHON ANALYSIS — CUSTOMER SEGMENTATION

4.1 RFM Methodology

- **Recency** — days since last order — lower is more engaged
- **Frequency** — number of distinct orders — higher is more loyal
- **Monetary** — total revenue generated — higher is higher value

4.2 Data Extraction & Preparation

Connect & Extract

```
from sqlalchemy import create_engine
import pymysql

engine = create_engine("mysql+pymysql://root:***@localhost/sales")

customers = pd.read_sql("select * from customers", engine)
orders = pd.read_sql("select * from orders", engine)
order_items = pd.read_sql("select * from order_items", engine)
```

Output: Connects Python to MySQL and loads the core tables into DataFrames. Establishes the live link between the database and the analysis notebook.

4.3 Basic Exploratory Data Analysis

Basic Dataset Summary

```
df.describe()
df.info()
df.value_counts()
```

Output: Displays statistical summaries, data types, missing values, and unique value counts of the dataset.

Customer Distribution by State

```
customers["state"].value_counts().plot(kind="bar")
plt.title("Customers by State")
plt.xlabel("State")
plt.ylabel("Number of Customers")
plt.show()
```

Output: Creates a bar chart showing the number of customers in each state. Helps identify regions with the highest customer concentration and supports location-based business decisions.

Order Status Distribution

```
orders["order_status"].value_counts().plot(kind="bar")
plt.title("Order Status")
plt.xlabel("Status")
plt.ylabel("Count")
plt.show()
```

Output: Displays the number of orders for each order status. Helps monitor order processing performance and identify delays or cancelled orders

Quantity Distribution

```
order_items["quantity"].plot(kind="hist", bins=10)
plt.title("Quantity Distribution")
plt.xlabel("Quantity")
plt.show()
```

Output: Creates a histogram showing the distribution of product quantities ordered. Helps understand customer purchasing patterns and the most common order quantities.

List Price Distribution

```
plt.boxplot(order_items["list_price"])
plt.title("List Price Boxplot")
plt.show()
```

Output: Displays the distribution of product list prices and highlights outliers. Helps identify unusually high or low product prices that may require further investigation

4.4 RFM Calculation & Segmentation

Merge & Set Snapshot Date

```
sales_data = pd.merge(orders, order_items, on="order_id")
sales_data["order_date"] = pd.to_datetime(sales_data["order_date"])
snapshot_date = sales_data["order_date"].max() + pd.Timedelta(days=1)
```

Output: Combines orders with order_items and defines a fixed reference date. Needed to measure Recency consistently across all customers.

Calculate & Save RFM Segments

```
rfm = sales_data.groupby("customer_id").agg({
    "order_date": lambda x: (snapshot_date - x.max()).days,
    "order_id": "nunique", "Total_price": "sum"
})
rfm.columns = ["Recency", "Frequency", "Monetary"]

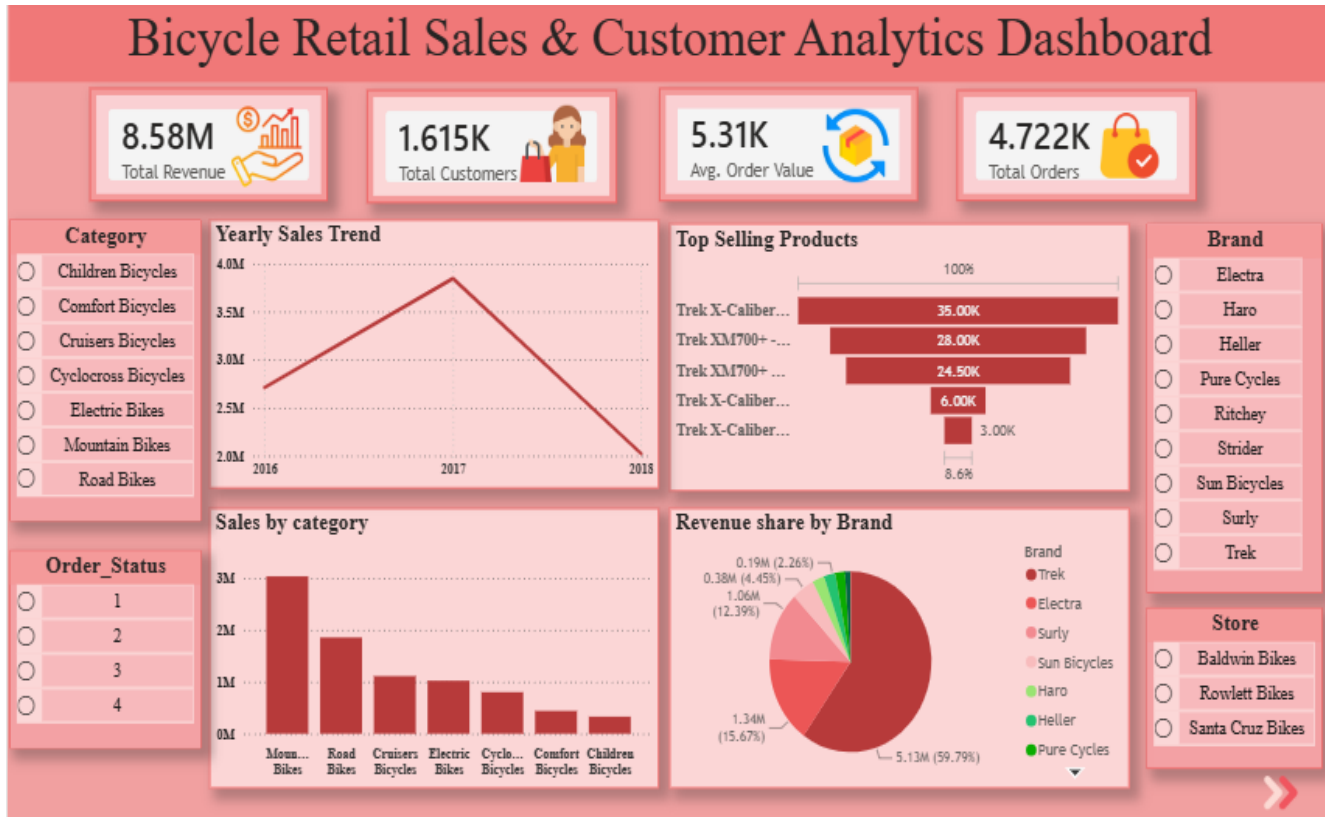
rfm.to_sql("customer_segments", con=engine, if_exists="replace", index=False)
rfm.to_csv("customer_segments.csv", index=False)
```

Output: Computes RFM per customer, then saves results to MySQL and CSV. This is the core output feeding the Customer Insights dashboard page.

CHAPTER 5

POWER BI DASHBOARD DESIGN

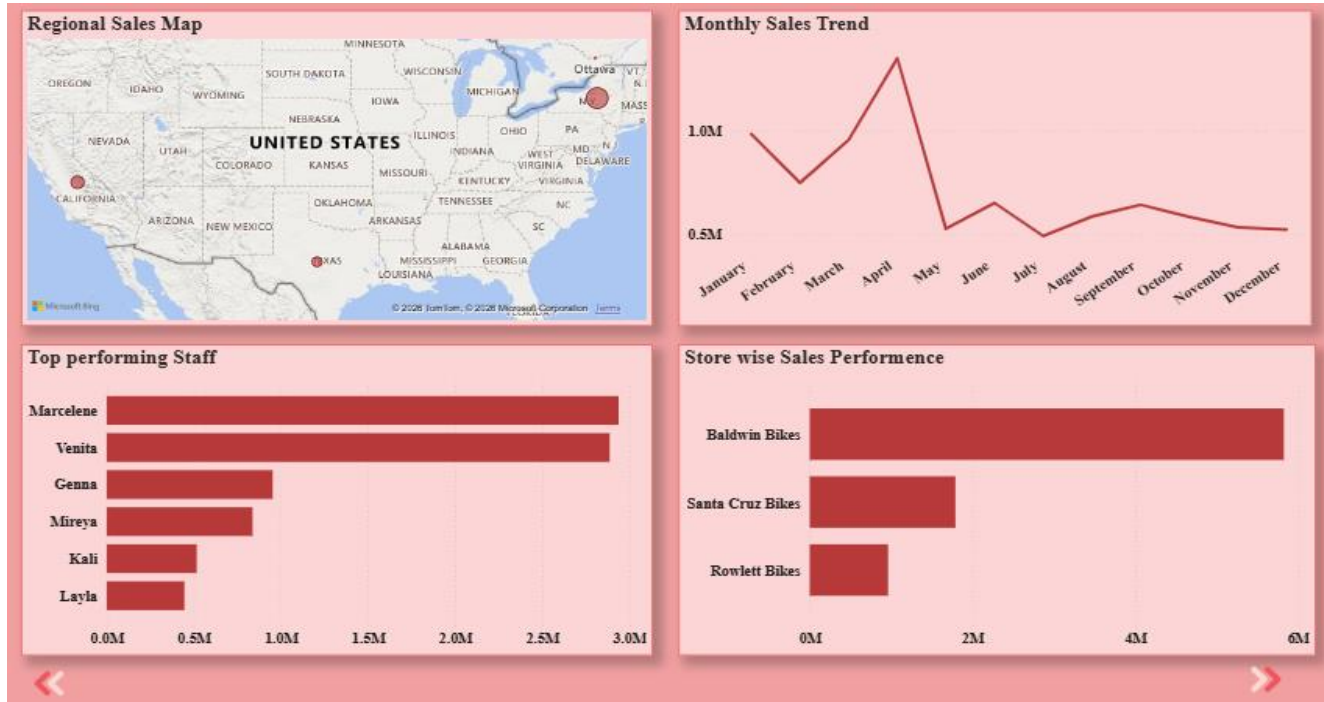
5.1 Overview Dashboard



Picture 1: Overview Dashboard

- KPI tiles: Total Revenue (8.58M), Total Customers (1,615), Avg. Order Value (5.31K), Total Orders (4,722).
- Yearly sales trend, top products, category and brand breakdown.
- Filters for category, store, brand, and order status.

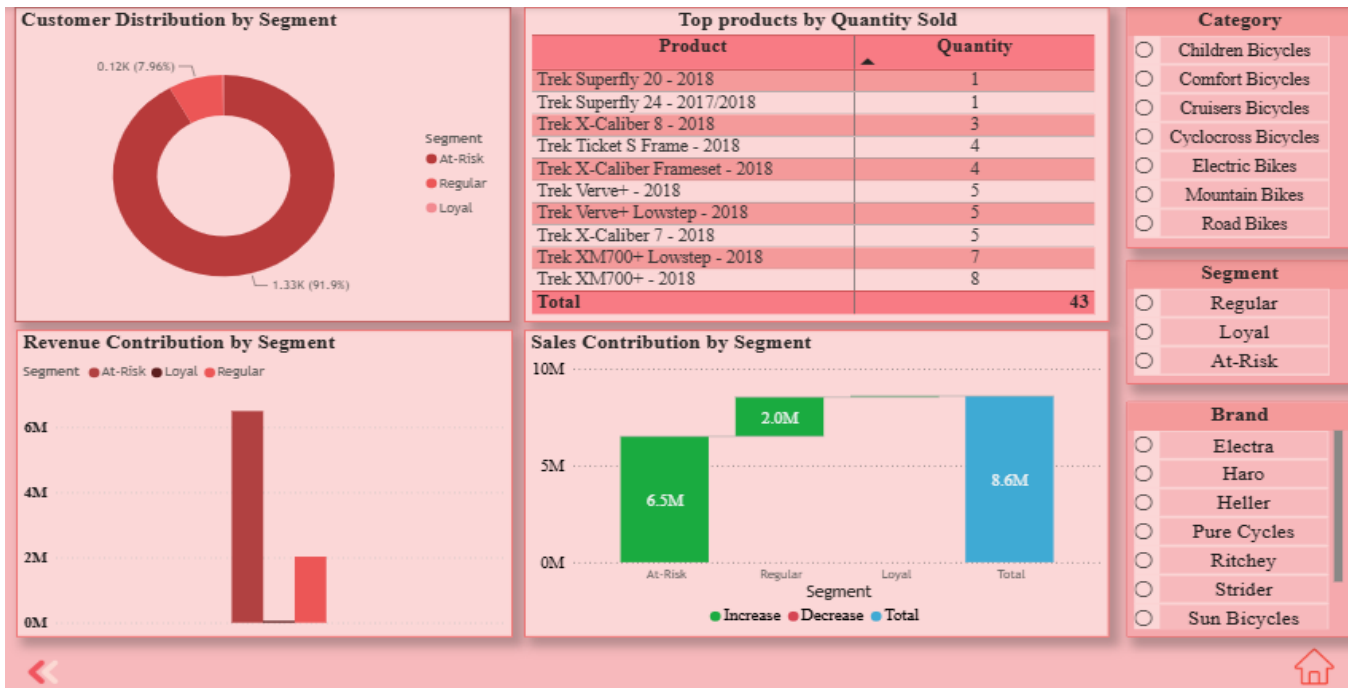
5.2 Sales Performance Dashboard



Picture 2: Sales Performance Dashboard

- Monthly sales trend, geographic sales map, staff performance, and store-wise revenue.

5.3 Customer Insights Dashboard



Picture 3: Customer Insights Dashboard

- Customer distribution by segment, revenue contribution by segment, and product-level quantity table.

CHAPTER 6

SYSTEM STUDY

6.1 Key Business Insights

- Baldwin Bikes is the top-performing store, notably ahead of the other two locations.
- Mountain Bikes is the highest-grossing category; Trek X-Caliber 8 – 2018 is the top product.
- Sales peaked in 2017 and declined in 2018 — this trend needs further investigation.
- April is a clear seasonal peak month for sales.
- Marcelene and Venita are the top-performing staff members by revenue.
- Regular customers form 91.9% of the base; the At-Risk segment (7.96%) is a targeted retention opportunity.
- Revenue contribution across segments is not proportional to segment size — smaller segments can be disproportionately valuable.

6.2 Conclusion

The analysis of this bicycle retail dataset provides insight into store performance, product trends, seasonal patterns, and customer value. SQL queries, Python-based RFM segmentation, and the Power BI dashboard together identify top-performing stores, products, and staff, and reveal actionable customer segments. This project demonstrates how a combined SQL + Python + Power BI workflow can be used for end-to-end retail analytics and decision support.

The findings help the business improve inventory planning, staffing decisions, and customer retention strategy. Overall, the project converts raw relational sales data into decision-ready business intelligence.

6.3 Future Enhancement

- Automate the SQL and Python pipeline to refresh customer segments on a scheduled basis.
- Incorporate a profit-margin measure alongside revenue for deeper analysis.
- Extend the RFM model with predictive churn analytics.
- Add year-over-year and month-over-month growth measures to the dashboard.